CENTRO UNIVERSITÁRIO – CATÓLICA DE SANTA CATARINA

ENGENHARIA DE SOFTWARE

RELATÓRIO:

COMPARAÇÃO DE ALGORITMOS DE BUSCA EM STRINGS

Alunos:
Mariele Vieira da Silva
Maria Alice Giuliari

1. Explicação dos Algoritmos Implementados e Arquitetura do Programa

Neste trabalho, foram implementados quatro algoritmos clássicos de busca de padrões em strings: Naive (Força Bruta), Rabin-Karp, Knuth-Morris-Pratt (KMP) e Boyer-Moore. Cada um deles possui abordagens diferentes para localizar ocorrências de um padrão dentro de um texto.

Conforme os requisitos de implementação, nós utilizamos o *Padrão Strategy* que define uma família de algoritmos, encapsula cada um deles e os torna substituíveis. Com isso, no projeto como um todo, todos os algoritmos de busca seguem a mesma interface da classe abstrata **`EstrategiadeBusca,`** herdando-a, depois definindo os atributos de nome e complexidade e implementado no método *buscar*.

Para a estruturação dos dados criamos duas classes: **`PassoExecucao`** que guarda tudo que acontece em uma comparação de algoritmo, também é utilizada para mostrar o passo a passo na tela e **`ResultadoBusca`** que contém métricas, posições encontradas e todos os passos, é responsável por trazer o resultado completo de uma execução de busca.

Naive (Força Bruta)

O algoritmo Naive é a abordagem mais simples para busca de padrões. Ele percorre o texto posição por posição, e, para cada posição, compara caractere por caractere com o padrão.

Se houver uma diferença, ele interrompe a comparação e avança uma posição no texto, reiniciando o processo.

- **Vantagens:**
  - É de fácil implementação.
  - Não requer pré-processamento.
- **Desvantagens:**
  - Há um alto número de comparações em textos grandes.
- **Complexidade:**
  - Melhor caso: O(n)
  - Caso médio: O(n·m)
  - Pior caso: O(n·m)
- **Exemplos:**
  - texto: A B C A B D
  - padrão: A B D
  - pos 0: A=A ✓, B=B ✓, C≠D ✗ → avança
  - pos 1: B≠A ✗ → avança
  - pos 2: C≠A ✗ → avança
  - pos 3: A=A ✓, B=B ✓, D=D ✓ → ENCONTROU em [3]!

```
class BuscaNaive(EstrategiaDeBusca):
    nome = "Naive"
    complexidade_melhor = "O(n)"
    complexidade_media = "O(n·m)"
```

```
    complexidade_pior = "O(n·m)"

    def buscar(self, texto: str, padrao: str) -> ResultadoBusca:
        n = len(texto)
        m = len(padrao)
        posicoes = []
        passos = []
        comparacoes = 0
        numero_passo = 0

        inicio = time.perf_counter()

        # Caso especial: texto ou padrão vazio → nada a buscar
        if m == 0 or n == 0:
            tempo = (time.perf_counter() - inicio) * 1000
                return self._montar_resultado(texto, padrao, [], 0, [],
tempo)

        # Para cada posição possível de início no texto...
        for inicio_janela in range(n - m + 1):

            # ...tenta casar o padrão inteiro a partir daqui
            j = 0
            while j < m:
                comparacoes += 1
                char_texto = texto[inicio_janela + j]
                char_padrao = padrao[j]
                houve_match = (char_texto == char_padrao)

                passos.append(PassoExecucao(
                    numero_passo=numero_passo,
                    posicao_texto=inicio_janela + j,
                    posicao_padrao=j,
                                    descricao=f"texto[{inicio_janela  +
j}]='{char_texto}' vs padrão[{j}]='{char_padrao}'",
                    houve_match=houve_match,
                                destaque_texto=list(range(inicio_janela,
inicio_janela + m)),
                    destaque_padrao=list(range(j + 1)),
                    dados_extras={"inicio_janela": inicio_janela},
                ))
                numero_passo += 1
```

```
                if not houve_match:
                    break  # falhou → tenta próxima janela
                j += 1

            if j == m:
                posicoes.append(inicio_janela)  # chegou até o fim →
encontrou!

        tempo = (time.perf_counter() - inicio) * 1000
            return self._montar_resultado(texto, padrao, posicoes,
comparacoes, passos, tempo)
```

algoritmos.py

Rabin-Karp

O algoritmo Rabin-Karp utiliza hashing para acelerar a busca. Ele calcula um valor hash para o padrão e para cada substring do texto com o mesmo tamanho.

Ao invés de comparar diretamente os caracteres, ele compara os hashes. Quando os hashes coincidem, uma verificação caractere por caractere é realizada para confirmar.

- **Vantagens:**
  - É eficiente para múltiplos padrões.
  - Reduz comparações diretas.
- **Desvantagens:**
  - Há a possibilidade de colisões de hash.
  - Pode degradar para força bruta.
- **Complexidade:**
  - Melhor caso: O(n + m)
  - Caso médio: O(n + m)
  - Pior caso: O(n·m)

```
# ALGORITMO 2: RABIN-KARP
class BuscaRabinKarp(EstrategiaDeBusca):
    nome = "Rabin-Karp"
    complexidade_melhor = "O(n+m)"
    complexidade_media = "O(n+m)"
    complexidade_pior = "O(n·m)"

        BASE = 256  # base do sistema posicional (tamanho do
alfabeto)
    MOD = 101  # módulo primo para manter o hash pequeno

    def buscar(self, texto: str, padrao: str) -> ResultadoBusca:
        n = len(texto)
        m = len(padrao)
```

```
        posicoes = []
        passos = []
        comparacoes = 0
        numero_passo = 0
            registro_hashes = []   # para exibir na aba "Tabelas
Internas"

        inicio = time.perf_counter()

        if m == 0 or n == 0 or m > n:
            tempo = (time.perf_counter() - inicio) * 1000
              return self._montar_resultado(texto, padrao, [], 0,
[], tempo)

        B = self.BASE
        MOD = self.MOD

          # h = B^(m-1) mod MOD → usado para remover o primeiro
caractere da janela
        h = pow(B, m - 1, MOD)

          # Calcula o hash inicial do padrão e da primeira janela
do texto
        hash_padrao = 0
        hash_janela = 0
        for i in range(m):
               hash_padrao = (B * hash_padrao + ord(padrao[i])) %
MOD
            hash_janela = (B * hash_janela + ord(texto[i])) % MOD

              registro_hashes.append({"janela": 0, "hash_texto":
hash_janela, "hash_padrao": hash_padrao})

        # Desliza a janela pelo texto
        for i in range(n - m + 1):
            hashes_iguais = (hash_janela == hash_padrao)

            info_extra = {
                "hash_texto": hash_janela,
                "hash_padrao": hash_padrao,
                "hashes_iguais": hashes_iguais,
                "inicio_janela": i,
            }
```

```
            if hashes_iguais:
                  # Hashes batem → verifica caractere a caractere
para confirmar
                for j in range(m):
                    comparacoes += 1
                    char_texto = texto[i + j]
                    char_padrao = padrao[j]
                    houve_match = (char_texto == char_padrao)

                    passos.append(PassoExecucao(
                        numero_passo=numero_passo,
                        posicao_texto=i + j,
                        posicao_padrao=j,
                            descricao=f"[Hash igual!] Confirmando:
texto[{i+j}]='{char_texto}' vs padrão[{j}]='{char_padrao}'",
                        houve_match=houve_match,
                        destaque_texto=list(range(i, i + m)),
                        destaque_padrao=list(range(j + 1)),
                        dados_extras=info_extra,
                    ))
                    numero_passo += 1

                    if not houve_match:
                          break  # foi uma colisão de hash → falso
positivo
                else:
                      posicoes.append(i)  # todos os chars batem →
encontrou!
            else:
                 # Hashes diferentes → podemos pular sem verificar
os chars
                passos.append(PassoExecucao(
                    numero_passo=numero_passo,
                    posicao_texto=i,
                    posicao_padrao=0,
                        descricao=f"Hash diferente na janela {i}:
texto={hash_janela} ≠ padrão={hash_padrao} → pula",
                    houve_match=False,
                    destaque_texto=list(range(i, i + m)),
                    destaque_padrao=[],
                    dados_extras=info_extra,
                ))
```

```
                numero_passo += 1

            # Atualiza o hash para a próxima janela (rolling hash
em O(1))
            if i < n - m:
                hash_janela = (B * (hash_janela - ord(texto[i]) *
h) + ord(texto[i + m])) % MOD
                if hash_janela < 0:
                    hash_janela += MOD
                        registro_hashes.append({"janela": i + 1,
"hash_texto": hash_janela, "hash_padrao": hash_padrao})

        tempo = (time.perf_counter() - inicio) * 1000
        tabelas = {
            "hashes": registro_hashes,
            "base": B,
            "mod": MOD,
            "hash_padrao": hash_padrao,
        }
           return self._montar_resultado(texto, padrao, posicoes,
comparacoes, passos, tempo, tabelas)
```

algoritmos.py

Knuth-Morris-Pratt (KMP)

O algoritmo KMP melhora a eficiência evitando comparações redundantes. Ele utiliza uma tabela chamada LPS (longest Prefix Suffix) que indica o quanto o padrão pode ser deslocado sem recomeçar do zero.

Isso permite que o algoritmo não volte no texto, apenas no padrão.

- **Vantagens:**
  - Evita retrocesso no texto.
  - É excelente para padrões com repetições.
- **Desvantagens:**
  - É mais complexo de implementar.
- **Complexidade:**
  - Melhor caso: O(n)
  - Caso médio: O(n + m)
  - Pior caso: O(n + m)

```
#  ALGORITMO 3: KNUTH-MORRIS-PRATT (KMP)
class BuscaKMP(EstrategiaDeBusca):
    nome = "KMP"
    complexidade_melhor = "O(n)"
    complexidade_media = "O(n+m)"
```

```
    complexidade_pior = "O(n+m)"

    def _construir_tabela_lps(self, padrao: str) -> List[int]:
        """
        Constrói a tabela LPS (Longest Proper Prefix-Suffix).
        Esta é a pré-computação do KMP, feita em O(m).
        """
        m = len(padrao)
         lps = [0] * m  # lps[0] é sempre 0 (prefixo próprio de 1 char =
vazio)

        comprimento = 0  # tamanho do prefixo atual
        i = 1

        while i < m:
            if padrao[i] == padrao[comprimento]:
                # Extendemos o prefixo-sufixo em mais um caractere
                comprimento += 1
                lps[i] = comprimento
                i += 1
            else:
                if comprimento != 0:
                    # Tentamos um prefixo menor (não retrocedem em i!)
                    comprimento = lps[comprimento - 1]
                else:
                    # Nenhum prefixo-sufixo possível
                    lps[i] = 0
                    i += 1

        return lps

    def buscar(self, texto: str, padrao: str) -> ResultadoBusca:
        n = len(texto)
        m = len(padrao)
        posicoes = []
        passos = []
        comparacoes = 0
        numero_passo = 0

        inicio = time.perf_counter()

        if m == 0 or n == 0:
            tempo = (time.perf_counter() - inicio) * 1000
```

```
                return self._montar_resultado(texto, padrao, [], 0, [],
tempo)

        lps = self._construir_tabela_lps(padrao)

        i = 0  # cursor no texto  (nunca retrocede!)
        j = 0  # cursor no padrão

        while i < n:
            comparacoes += 1
            char_texto = texto[i]
            char_padrao = padrao[j]
            houve_match = (char_texto == char_padrao)

            # Calcula o salto que faríamos se falhar aqui
             salto_lps = lps[j - 1] if (not houve_match and j > 0) else
None

            passos.append(PassoExecucao(
                numero_passo=numero_passo,
                posicao_texto=i,
                posicao_padrao=j,
                               descricao=f"texto[{i}]='{char_texto}'  vs
padrão[{j}]='{char_padrao}' | LPS[{j}]={lps[j]}",
                houve_match=houve_match,
                destaque_texto=[i],
                destaque_padrao=[j],
                         dados_extras={"lps": lps[:], "i": i, "j": j,
"salto_lps": salto_lps},
            ))
            numero_passo += 1

            if houve_match:
                i += 1  # avança nos dois
                j += 1
            else:
                if j != 0:
                    # Usa o LPS para saltar → não retrocede i!
                    j = lps[j - 1]
                else:
                       i += 1  # nem o primeiro char casou → avança no
texto
```

```
            # Chegou ao fim do padrão → encontrou!
            if j == m:
                posicoes.append(i - j)
                      j = lps[j - 1]   # prepara para buscar a próxima
ocorrência

        tempo = (time.perf_counter() - inicio) * 1000
        tabelas = {
            "lps": [
                      {"indice": idx, "char": padrao[idx], "valor_lps":
lps[idx]}
                for idx in range(m)
            ],
        }
               return  self._montar_resultado(texto,  padrao,  posicoes,
comparacoes, passos, tempo, tabelas)
```

algoritmos.py

Boyer-Moore

O algoritmo Boyer-Moore realiza comparações da direita para a esquerda e utiliza heurísticas para pular partes do texto.

Neste trabalho, utilizamos a heurística de Bad Character, que permite saltos maiores caso ocorra uma falha.

- **Vantagens:**
  - É muito eficiente na prática.
  - Realiza menos comparações.
- **Desvantagens:**
  - Sua implementação é mais complexa.
- **Complexidade:**
  - Melhor caso: O(n/m)
  - Caso médio: O(n)
  - Pior caso: O(n·m)

```
#  ALGORITMO 4: BOYER-MOORE
class BuscaBoyerMoore(EstrategiaDeBusca):
    nome = "Boyer-Moore"
    complexidade_melhor = "O(n/m)"
    complexidade_media = "O(n)"
    complexidade_pior = "O(n·m)"

    def _construir_tabela_mau_caractere(self, padrao: str) -> Dict[str,
int]:
        """
        Tabela de mau caractere: char → último índice no padrão.
```

```
          Chars ausentes retornam -1 quando consultados com .get(char,
-1).
        """
        tabela = {}
        for i, char in enumerate(padrao):
            tabela[char] = i  # sobrescreve → fica só o último índice
        return tabela

    def buscar(self, texto: str, padrao: str) -> ResultadoBusca:
        n = len(texto)
        m = len(padrao)
        posicoes = []
        passos = []
        comparacoes = 0
        numero_passo = 0

        inicio = time.perf_counter()

        if m == 0 or n == 0 or m > n:
            tempo = (time.perf_counter() - inicio) * 1000
                return self._montar_resultado(texto, padrao, [], 0, [],
tempo)

        tabela_mc = self._construir_tabela_mau_caractere(padrao)

          deslocamento = 0  # quantas posições o padrão está deslocado
sobre o texto

        while deslocamento <= n - m:

            # Compara da direita para a esquerda
            j = m - 1

            while j >= 0:
                comparacoes += 1
                char_texto = texto[deslocamento + j]
                char_padrao = padrao[j]
                houve_match = (char_padrao == char_texto)

                # Índice do mau caractere no padrão (-1 se não existir)
                indice_mc = tabela_mc.get(char_texto, -1)
                salto = max(1, j - indice_mc) if not houve_match else 0
```

```
                passos.append(PassoExecucao(
                    numero_passo=numero_passo,
                    posicao_texto=deslocamento + j,
                    posicao_padrao=j,
                    descricao=(
                            f"texto[{deslocamento+j}]='{char_texto}' vs
padrão[{j}]='{char_padrao}' "
                        f"(direita→esquerda)"
                    ),
                    houve_match=houve_match,
                                destaque_texto=list(range(deslocamento,
deslocamento + m)),
                    destaque_padrao=[j],
                    dados_extras={
                        "deslocamento": deslocamento,
                        "indice_mc": indice_mc,
                        "salto": salto,
                        "mau_caractere": char_texto,
                    },
                ))
                numero_passo += 1

                if not houve_match:
                      break  # achou o mau caractere → calcula salto e
pula
                j -= 1

            if j < 0:
                # j chegou a -1 → casou tudo → encontrou!
                posicoes.append(deslocamento)
                   prox = tabela_mc.get(texto[deslocamento + m], -1) if
deslocamento + m < n else -1
                deslocamento += m - prox
            else:
                # Aplica a heurística do mau caractere
                indice_mc = tabela_mc.get(texto[deslocamento + j], -1)
                deslocamento += max(1, j - indice_mc)

        tempo = (time.perf_counter() - inicio) * 1000
        tabelas = {
            "mau_caractere": [
                {"char": char, "ultimo_indice": idx}
                for char, idx in sorted(tabela_mc.items())
```

```
            ],
        }
            return self._montar_resultado(texto, padrao, posicoes,
comparacoes, passos, tempo, tabelas)
```

algoritmos.py

2. Análise Comparativa de Desempenho

A aplicação desenvolvida permite a comparação de algoritmos com base em:

- Tempo de execução (ms)
- Número de comparações realizadas
- Número de ocorrências encontradas

2.1 - Observações gerais

- O **Naive** apresentou maior número de comparações, especialmente em textos maiores.
- O **Rabin-Karp** reduziu comparações quando os hashes não coincidiam, mas ainda precisou validar possíveis matches.
- O **KMP** teve desempenho estável, mantendo baixa quantidade de comparações mesmo em casos com repetição.
- O **Boyer-Moore** foi o mais eficiente na maioria dos cenários, realizando menos comparações devido aos saltos.

Algoritmo	Tempo (ms)	Comparações	Ocorrências	Melhor	Médio	Pior
Naive	2.9 ms	1.120	0	O(n)	O(n·m)	O(n·m)
Rabin-Karp	1.7 ms	9	0	O(n+m)	O(n+m)	O(n·m)
KMP	4.4 ms	1.018	0	O(n)	O(n+m)	O(n+m)
Boyer-Moore	0.4 ms 🏆	285	0	O(n/m)	O(n)	O(n·m)

2.2 - Conclusão da Comparação:

- **Melhor desempenho geral:** Boyer-Moore
- **Melhor estabilidade:** KMP
- **Melhor para múltiplos padrões:** Rabin-Karp

- **Mais simples:** Naive

3. Discussão: Quando cada algoritmo é mais eficiente

A escolha do algoritmo ideal depende do contexto em que será usada. No geral, podemos dizer que:

Naive

Indicado para:

- Textos pequenos
- Situações simples ou didáticas
- Quando a simplicidade é a prioridade

Rabin-Karp

Indicado para:

- Busca de múltiplos padrões
- Aplicações com uso de hashing
- Casos onde colisões são raras

KMP

Indicado para:

- Textos com muitas repetições
- Quando é importante evitar retrabalho
- Processamento eficiente em tempo linear

Boyer-Moore

Indicado para:

- Textos longos
- Casos reais (busca em arquivos, editores, etc)
- Quando se deseja maior desempenho prático

4. Conclusão Geral

Os algoritmos de busca em strings apresentam diferentes estratégias para resolver o mesmo problema. Enquanto o Naive é simples e direto, outros algoritmos, como o KMP e o Boyer-Moore, utilizam tanto pré-processamento quanto heurísticas para melhorar significativamente o desempenho.

Na prática, o Boyer-Moore tende a ser o mais eficiente, enquanto o KMP garante desempenho linear consistente. Já o Rabin-karp se destaca em cenários específicos com múltiplos padrões.